



UTT

UNIVERSIDAD TECNOLÓGICA DE TIJUANA

GOBIERNO DE BAJA CALIFORNIA

TEMA

Secure Coding Principles Specification

PRESENTADO POR

Lopez Domínguez Luis Enrique

GRUPO

10° B

MATERIA

Desarrollo móvil integral

PROFESOR

Ray Parra

Tijuana, Baja California, 15 de enero del 2025

Instructions:

1. Search different sources of information for the Secure Coding Principles Specification topic
2. Read and Understand the topic
3. Make a summary of the topic
4. Convert the summary to PDF
5. Upload the PDF to the corresponding assignment
6. Wait for feedback from the teacher

Lista de verificación de prácticas de codificación segura

Validación de entrada

- Realice toda la validación de entrada en un sistema confiable (del lado del servidor, no del lado del cliente)
- Identificar todas las fuentes de datos y clasificarlas en confiables y no confiables.
- Validar todos los datos de fuentes no confiables (bases de datos, flujos de archivos, etc.)
- Utilice una rutina de validación de entrada centralizada para toda la aplicación
- Especificar conjuntos de caracteres, como UTF-8, para todas las fuentes de entrada (canonización)
- Codifique la entrada en un conjunto de caracteres común antes de validar
- Todos los errores de validación deberían resultar en el rechazo de la entrada.
- Si el sistema admite conjuntos de caracteres extendidos UTF-8 y valida después de que se complete la decodificación UTF-8
- Validar todos los datos proporcionados por el cliente antes de procesarlos
- Verifique que los valores del encabezado del protocolo tanto en las solicitudes como en las respuestas contengan solo caracteres ASCII
- Validar datos de redirecciones
- Validar los tipos de datos esperados utilizando una lista de "permisos" en lugar de una lista de "denegaciones"
- Validar rango de datos
- Validar la longitud de los datos
- Si se debe permitir alguna entrada potencialmente peligrosa, se deben implementar controles adicionales.

- Si la rutina de validación estándar no puede abordar algunas entradas, utilice verificaciones discretas adicionales
- Utilice la canonización para abordar los ataques de ofuscación

Codificación de salida

- Realice toda la codificación de salida en un sistema confiable (del lado del servidor, no del lado del cliente)
- Utilice una rutina estándar y probada para cada tipo de codificación saliente
- Especifique conjuntos de caracteres, como UTF-8, para todas las salidas
- Codifique contextualmente todos los datos devueltos al cliente desde fuentes no confiables
- Asegúrese de que la codificación de salida sea segura para todos los sistemas de destino
- Sanitizar contextualmente toda la salida de datos no confiables para consultas de SQL, XML y LDAP
- Sanitizar toda la salida de datos no confiables a los comandos del sistema operativo

Autenticación y gestión de contraseñas

- Requerir autenticación para todas las páginas y recursos, excepto aquellos específicamente destinados a ser públicos
- Todos los controles de autenticación deben implementarse en un sistema confiable
- Establecer y utilizar servicios de autenticación estándar y probados siempre que sea posible
- Utilice una implementación centralizada para todos los controles de autenticación, incluidas las bibliotecas que llaman a servicios de autenticación externos

- Separe la lógica de autenticación del recurso que se solicita y utilice la redirección hacia y desde el control de autenticación centralizado.
- Todos los controles de autenticación deberían fallar de forma segura
- Todas las funciones administrativas y de gestión de cuentas deben ser al menos tan seguras como el mecanismo de autenticación principal.
- Si su aplicación administra un almacén de credenciales, utilice hashes salados unidireccionales criptográficamente fuertes
- El hash de contraseñas debe implementarse en un servidor de sistema confiable, no en el lado del cliente)
- Validar los datos de autenticación solo después de completar todos los datos ingresados
- Las respuestas de error de autenticación no deben indicar qué parte de los datos de autenticación era incorrecta
- Utilice la autenticación para conexiones a sistemas externos que involucran información o funciones confidenciales
- Las credenciales de autenticación para acceder a servicios externos a la aplicación deben almacenarse en un almacén seguro.
- Utilice únicamente solicitudes HTTP POST para transmitir credenciales de autenticación
- Envíe contraseñas no temporales únicamente a través de una conexión cifrada o como datos cifrados
- Hacer cumplir los requisitos de complejidad de contraseñas establecidos por políticas o regulaciones
- Hacer cumplir los requisitos de longitud de contraseña establecidos por política o regulación
- La entrada de contraseña debe quedar oculta en la pantalla del usuario.

- Imponer la desactivación de la cuenta después de una cantidad establecida de intentos de inicio de sesión no válidos
- Las operaciones de restablecimiento y cambio de contraseña requieren el mismo nivel de controles que la creación y autenticación de cuentas.
- Las preguntas de restablecimiento de contraseña deben admitir respuestas suficientemente aleatorias
- Si utiliza restablecimientos basados en correo electrónico, solo envíe correos electrónicos a una dirección registrada previamente con un enlace/contraseña temporal
- Las contraseñas y enlaces temporales deben tener un tiempo de expiración corto
- Hacer cumplir el cambio de contraseñas temporales en el próximo uso
- Notificar a los usuarios cuando se produce un restablecimiento de contraseña
- Evitar la reutilización de contraseñas
- Las contraseñas deben tener al menos un día de antigüedad antes de poder cambiarlas, para evitar ataques por reutilización de contraseñas.
- Aplicar cambios de contraseñas según los requisitos establecidos en la política o regulación, con el tiempo entre restablecimientos controlado administrativamente
- Desactivar la función "recordarme" para los campos de contraseña
- El último uso (exitoso o no) de una cuenta de usuario debe informarse al usuario en su próximo inicio de sesión exitoso.
- Implementar monitoreo para identificar ataques contra múltiples cuentas de usuario, utilizando la misma contraseña
- Cambie todas las contraseñas y los ID de usuario predeterminados proporcionados por el proveedor o deshabilite las cuentas asociadas

- Vuelva a autenticar a los usuarios antes de realizar operaciones críticas
- Utilice la autenticación multifactor para cuentas transaccionales de alto valor o altamente sensibles
- Si utiliza un código de terceros para la autenticación, inspeccione el código cuidadosamente para asegurarse de que no esté afectado por ningún código malicioso.

Gestión de sesiones

- Utilice los controles de administración de sesiones del servidor o del marco de trabajo. La aplicación debe reconocer únicamente estos identificadores de sesión como válidos.
- La creación del identificador de sesión siempre debe realizarse en un sistema confiable (del lado del servidor, no del lado del cliente)
- Los controles de gestión de sesiones deben utilizar algoritmos bien examinados que garanticen identificadores de sesión suficientemente aleatorios.
- Establezca el dominio y la ruta para las cookies que contienen identificadores de sesión autenticados en un valor restringido apropiado para el sitio
- La función de cierre de sesión debe finalizar por completo la sesión o conexión asociada
- La funcionalidad de cierre de sesión debe estar disponible en todas las páginas protegidas por autorización
- Establecer un tiempo de inactividad de sesión lo más breve posible, en función del equilibrio entre el riesgo y los requisitos funcionales del negocio.
- No permitir inicios de sesión persistentes y aplicar terminaciones periódicas de sesión, incluso cuando la sesión esté activa
- Si se estableció una sesión antes de iniciar sesión, cierre esa sesión y establezca una nueva sesión después de un inicio de sesión exitoso

- Generar un nuevo identificador de sesión en cualquier nueva autenticación
- No permitir inicios de sesión simultáneos con el mismo ID de usuario
- No exponga identificadores de sesión en URL, mensajes de error o registros
- Implementar controles de acceso adecuados para proteger los datos de la sesión del lado del servidor del acceso no autorizado por parte de otros usuarios del servidor.
- Generar un nuevo identificador de sesión y desactivar el antiguo periódicamente
- Generar un nuevo identificador de sesión si la seguridad de la conexión cambia de HTTP a HTTPS, como puede ocurrir durante la autenticación
- Utilice HTTPS de forma constante en lugar de cambiar entre HTTP y HTTPS
- Complemente la gestión de sesiones estándar para operaciones sensibles del lado del servidor, como la gestión de cuentas, mediante el uso de parámetros o tokens aleatorios fuertes por sesión.
- Complemente la gestión de sesiones estándar para operaciones altamente sensibles o críticas mediante el uso de tokens o parámetros aleatorios fuertes por solicitud, en lugar de por sesión.
- Establezca el atributo "seguro" para las cookies transmitidas a través de una conexión TLS
- Establezca cookies con el atributo HttpOnly, a menos que requiera específicamente que los scripts del lado del cliente dentro de su aplicación lean o establezcan el valor de una cookie

Control de acceso

- Utilice únicamente objetos de sistema confiables, por ejemplo, objetos de sesión del lado del servidor, para tomar decisiones de autorización de acceso.

- Utilice un único componente para todo el sitio para comprobar la autorización de acceso. Esto incluye bibliotecas que llaman a servicios de autorización externos
- Los controles de acceso deben fallar de forma segura
- Denegar todo acceso si la aplicación no puede acceder a su información de configuración de seguridad
- Aplicar controles de autorización en cada solicitud, incluidas aquellas realizadas mediante scripts del lado del servidor
- Separar la lógica privilegiada del resto del código de la aplicación
- Restringir el acceso a archivos u otros recursos, incluidos aquellos fuera del control directo de la aplicación, solo a usuarios autorizados
- Restringir el acceso a URL protegidas únicamente a usuarios autorizados
- Restringir el acceso a funciones protegidas únicamente a usuarios autorizados
- Restringir las referencias directas a objetos únicamente a usuarios autorizados
- Restringir el acceso a los servicios únicamente a los usuarios autorizados
- Restringir el acceso a los datos de la aplicación únicamente a los usuarios autorizados
- Restringir el acceso a los atributos de usuario y datos y a la información de políticas utilizada por los controles de acceso
- Restringir el acceso a la información de configuración relevante para la seguridad solo a los usuarios autorizados
- Las representaciones de las reglas de control de acceso en la capa de presentación y la implementación del lado del servidor deben coincidir

- Si los datos de estado deben almacenarse en el cliente, utilice el cifrado y la verificación de integridad en el lado del servidor para detectar alteraciones del estado.
- Imponer flujos lógicos de aplicaciones para cumplir con las reglas comerciales
- Limitar la cantidad de transacciones que un solo usuario o dispositivo puede realizar en un período de tiempo determinado, lo suficientemente baja como para disuadir ataques automatizados, pero por encima de los requisitos comerciales reales.
- Utilice el encabezado "referer" solo como una verificación complementaria, nunca debe ser la única verificación de autorización ya que puede ser falsificada.
- Si se permiten sesiones autenticadas prolongadas, vuelva a validar periódicamente la autorización de un usuario para asegurarse de que sus privilegios no hayan cambiado y, si así fuera, cierre la sesión del usuario y obíguelo a volver a autenticarse.
- Implementar la auditoría de cuentas y hacer cumplir la desactivación de cuentas no utilizadas
- La aplicación debe permitir la desactivación de cuentas y la finalización de sesiones cuando cesa la autorización.
- Las cuentas de servicio o las cuentas que admiten conexiones hacia o desde sistemas externos deben tener el menor privilegio posible.
- Cree una política de control de acceso para documentar las reglas de negocio, los tipos de datos y los criterios y/o procesos de autorización de acceso de una aplicación, de modo que el acceso pueda aprovisionarse y controlarse adecuadamente. Esto incluye la identificación de los requisitos de acceso tanto para los datos como para los recursos del sistema.

Prácticas criptográficas

- Todas las funciones criptográficas utilizadas para proteger los secretos del usuario de la aplicación deben implementarse en un sistema confiable.
- Proteger los secretos del acceso no autorizado
- Los módulos criptográficos deben fallar de forma segura
- Todos los números aleatorios, nombres de archivos aleatorios, GUID aleatorios y cadenas aleatorias deben generarse utilizando el generador de números aleatorios aprobado por el módulo criptográfico.
- Los módulos criptográficos utilizados por la aplicación deben ser compatibles con FIPS 140-2 o un estándar equivalente
- Establecer y utilizar una política y un proceso sobre cómo se gestionarán las claves criptográficas.

Manejo y registro de errores

- No divulgue información confidencial en respuestas de error, incluidos detalles del sistema, identificadores de sesión o información de la cuenta.
- Utilice controladores de errores que no muestren información de depuración o seguimiento de pila
- Implementar mensajes de error genéricos y utilizar páginas de error personalizadas
- La aplicación debe gestionar los errores de la aplicación y no depender de la configuración del servidor.
- Liberar correctamente la memoria asignada cuando se producen condiciones de error
- La lógica de manejo de errores asociada con los controles de seguridad debe denegar el acceso de forma predeterminada
- Todos los controles de registro deben implementarse en un sistema confiable

- Los controles de registro deben respaldar tanto el éxito como el fracaso de eventos de seguridad específicos
- Asegúrese de que los registros contengan datos importantes de eventos de registro
- Asegúrese de que las entradas de registro que incluyan datos no confiables no se ejecuten como código en la interfaz o el software de visualización de registros previstos
- Restringir el acceso a los registros únicamente a personas autorizadas
- Utilice una rutina central para todas las operaciones de registro
- No almacene información confidencial en registros, incluidos detalles innecesarios del sistema, identificadores de sesión o contraseñas.
- Asegúrese de que exista un mecanismo para realizar análisis de registros
- Registrar todos los fallos de validación de entrada
- Registrar todos los intentos de autenticación, especialmente los fallos
- Registrar todos los fallos de control de acceso
- Registrar todos los eventos de manipulación aparente, incluidos los cambios inesperados en los datos de estado
- Registrar intentos de conexión con tokens de sesión no válidos o vencidos
- Registrar todas las excepciones del sistema
- Registrar todas las funciones administrativas, incluidos los cambios en la configuración de seguridad
- Registrar todos los fallos de conexión TLS del backend
- Registrar fallos del módulo criptográfico
- Utilice una función hash criptográfica para validar la integridad de las entradas del registro

Protección de datos

- Implementar el mínimo privilegio, restringir a los usuarios únicamente a la funcionalidad, los datos y la información del sistema que se requieren para realizar sus tareas.
- Proteja todas las copias temporales o en caché de datos confidenciales almacenados en el servidor contra accesos no autorizados y elimine esos archivos de trabajo temporales tan pronto como ya no sean necesarios.
- Cifrar información almacenada altamente confidencial, como datos de verificación de autenticación, incluso si están en el lado del servidor
- Proteger el código fuente del lado del servidor para que no sea descargado por un usuario
- No almacene contraseñas, cadenas de conexión u otra información confidencial en texto claro o de cualquier manera que no sea criptográficamente segura en el lado del cliente.
- Eliminar comentarios en el código de producción accesible para el usuario que puedan revelar información confidencial del sistema backend u otra información
- Elimine la documentación innecesaria de la aplicación y del sistema, ya que puede revelar información útil a los atacantes.
- No incluya información confidencial en los parámetros de la solicitud HTTP GET
- Deshabilite las funciones de autocompletar en formularios que se espera que contengan información confidencial, incluida la autenticación
- Deshabilitar el almacenamiento en caché del lado del cliente en páginas que contienen información confidencial
- La aplicación debe admitir la eliminación de datos confidenciales cuando dichos datos ya no sean necesarios.

- Implemente controles de acceso adecuados para los datos confidenciales almacenados en el servidor. Esto incluye datos en caché, archivos temporales y datos a los que solo deben tener acceso usuarios específicos del sistema.

Seguridad de las comunicaciones

- Implementar el cifrado para la transmisión de toda la información confidencial. Esto debería incluir TLS para proteger la conexión y puede complementarse con cifrado discreto de archivos confidenciales o conexiones que no se basen en HTTP
- Los certificados TLS deben ser válidos y tener el nombre de dominio correcto, no estar vencidos y deben instalarse con certificados intermedios cuando sea necesario.
- Las conexiones TLS fallidas no deberían recurrir a una conexión insegura
- Utilice conexiones TLS para todo el contenido que requiera acceso autenticado y para toda otra información confidencial.
- Utilice TLS para conexiones a sistemas externos que involucren información o funciones confidenciales
- Utilice una única implementación TLS estándar que esté configurada adecuadamente
- Especificar codificaciones de caracteres para todas las conexiones
- Filtrar parámetros que contienen información confidencial del referente HTTP, al vincularse a sitios externos

Configuración del sistema

- Asegúrese de que los servidores, los marcos y los componentes del sistema estén ejecutando la última versión aprobada
- Asegúrese de que los servidores, los marcos y los componentes del sistema tengan todos los parches emitidos para la versión en uso

- Desactivar listados de directorios
- Restrinja las cuentas de servidor web, procesos y servicios a los menores privilegios posibles
- Cuando se produzcan excepciones, fallar de forma segura
- Eliminar todas las funciones y archivos innecesarios
- Eliminar el código de prueba o cualquier funcionalidad que no esté destinada a la producción, antes de la implementación.
- Evite la divulgación de la estructura de su directorio en el archivo robots.txt colocando directorios que no están destinados a la indexación pública en un directorio principal aislado
- Define qué métodos HTTP, Get o Post, admitirá la aplicación y si se manejarán de manera diferente en diferentes páginas de la aplicación.
- Deshabilitar métodos HTTP innecesarios
- Si el servidor web maneja diferentes versiones de HTTP, asegúrese de que estén configuradas de manera similar y asegúrese de que se comprendan las diferencias.
- Eliminar información innecesaria de los encabezados de respuesta HTTP relacionados con el sistema operativo, la versión del servidor web y los marcos de aplicación
- El almacén de configuración de seguridad de la aplicación debe poder generarse en formato legible para humanos para respaldar la auditoría.
- Implementar un sistema de gestión de activos y registrar los componentes del sistema y el software en él.
- Aislar los entornos de desarrollo de la red de producción y proporcionar acceso solo a los grupos de desarrollo y prueba autorizados

- Implementar un sistema de control de cambios de software para gestionar y registrar cambios en el código tanto en desarrollo como en producción.

Seguridad de la base de datos

- Utilice consultas parametrizadas fuertemente tipadas
- Utilice la validación de entrada y la codificación de salida y asegúrese de tener en cuenta los metacaracteres. Si esto falla, no ejecute el comando de base de datos.
- Asegúrese de que las variables estén fuertemente tipadas
- La aplicación debe utilizar el nivel de privilegio más bajo posible al acceder a la base de datos.
- Utilice credenciales seguras para acceder a la base de datos
- Las cadenas de conexión no deben estar codificadas en la aplicación. Deben almacenarse en un archivo de configuración independiente en un sistema confiable y deben estar cifradas.
- Utilice procedimientos almacenados para abstraer el acceso a los datos y permitir la eliminación de permisos para las tablas base en la base de datos.
- Cerrar la conexión lo antes posible
- Eliminar o cambiar todas las contraseñas administrativas de bases de datos predeterminadas
- Desactivar todas las funciones innecesarias de la base de datos
- Eliminar contenido innecesario del proveedor predeterminado (por ejemplo, esquemas de muestra)
- Deshabilite cualquier cuenta predeterminada que no sea necesaria para cumplir con los requisitos comerciales

- La aplicación debe conectarse a la base de datos con diferentes credenciales para cada distinción de confianza (por ejemplo, usuario, usuario de solo lectura, invitado, administradores)

Gestión de archivos

- No pase datos proporcionados por el usuario directamente a ninguna función de inclusión dinámica
- Requerir autenticación antes de permitir que se cargue un archivo
- Limite el tipo de archivos que se pueden cargar únicamente a aquellos que sean necesarios para fines comerciales.
- Valide que los archivos cargados sean del tipo esperado verificando los encabezados de archivo en lugar de la extensión del archivo
- No guarde archivos en el mismo contexto web que la aplicación
- Impedir o restringir la carga de cualquier archivo que pueda ser interpretado por el servidor web.
- Desactivar privilegios de ejecución en directorios de carga de archivos
- Implemente una carga segura en UNIX montando el directorio de archivos de destino como una unidad lógica utilizando la ruta asociada o el entorno chroot
- Al hacer referencia a archivos existentes, utilice una lista de nombres y tipos de archivos permitidos
- No pase datos proporcionados por el usuario a una redirección dinámica
- No pase rutas de directorios o archivos, utilice valores de índice asignados a una lista predefinida de rutas
- Nunca envíe la ruta absoluta del archivo al cliente
- Asegúrese de que los archivos y recursos de la aplicación sean de solo lectura

- Analizar los archivos cargados por el usuario en busca de virus y malware

Gestión de la memoria

- Utilice controles de entrada y salida para datos no confiables
- Compruebe que el buffer sea tan grande como el especificado
- Al utilizar funciones que aceptan una cantidad de bytes, asegúrese de que la terminación NULL se gestione correctamente
- Verifique los límites del búfer si se llama a la función en un bucle y protéjase contra el desbordamiento
- Trunca todas las cadenas de entrada a una longitud razonable antes de pasarlas a otras funciones
- Cierra específicamente los recursos, no confíes en la recolección de basura
- Utilice pilas no ejecutables cuando estén disponibles
- Evite el uso de funciones vulnerables conocidas
- Libere adecuadamente la memoria asignada al finalizar las funciones y en todos los puntos de salida
- Sobrescriba cualquier información confidencial almacenada en la memoria asignada en todos los puntos de salida de la función

Prácticas generales de codificación

- Utilice código administrado probado y aprobado en lugar de crear código nuevo no administrado para tareas comunes
- Utilice API integradas específicas de cada tarea para realizar tareas del sistema operativo. No permita que la aplicación envíe comandos directamente al sistema operativo, especialmente mediante el uso de shells de comandos iniciados por la aplicación.

- Utilice sumas de comprobación o hashes para verificar la integridad del código interpretado, las bibliotecas, los ejecutables y los archivos de configuración.
- Utilice el bloqueo para evitar múltiples solicitudes simultáneas o utilice un mecanismo de sincronización para evitar condiciones de carrera
- Proteja las variables y los recursos compartidos del acceso concurrente inapropiado
- Inicialice explícitamente todas sus variables y otros almacenes de datos, ya sea durante la declaración o justo antes del primer uso
- En los casos en que la aplicación debe ejecutarse con privilegios elevados, aumente los privilegios lo más tarde posible y elimínelos lo antes posible.
- Evite errores de cálculo al comprender la representación subyacente de su lenguaje de programación
- No pase datos proporcionados por el usuario a ninguna función de ejecución dinámica
- Restringir que los usuarios generen código nuevo o alteren código existente
- Revisar todas las aplicaciones secundarias, el código de terceros y las bibliotecas para determinar la necesidad comercial y validar la funcionalidad segura.

Resumen

Las prácticas de codificación segura son principios y lineamientos que se aplican durante el desarrollo de software para prevenir vulnerabilidades y proteger las aplicaciones frente a amenazas de seguridad. Estas prácticas abarcan áreas clave como validación de datos, control de accesos, manejo de errores, seguridad de las comunicaciones y más. A continuación, se resumen las principales prácticas:

1. Validación de Entradas

Verificar y limpiar los datos proporcionados por el usuario para asegurarse de que cumplan con los formatos, longitudes y tipos esperados, previniendo ataques como inyecciones de código.

2. Codificación de Salidas

Asegurar que las respuestas del servidor estén codificadas de manera segura para evitar vulnerabilidades como Cross-Site Scripting (XSS), utilizando estándares como UTF-8.

3. Autenticación y Gestión de Contraseñas

Implementar métodos robustos de autenticación, como contraseñas fuertes y autenticación multifactor, almacenando credenciales de forma segura mediante hashes como bcrypt.

4. Gestión de Sesiones

Proteger las sesiones generando identificadores únicos y seguros, estableciendo tiempos de expiración y utilizando cookies con configuraciones seguras (HttpOnly y Secure).

5. Control de Acceso

Definir políticas claras de permisos para garantizar que los usuarios solo accedan a los recursos asignados y prevenir la escalación de privilegios.

6. Prácticas Criptográficas

Usar algoritmos de cifrado modernos y evitar algoritmos obsoletos como MD5. Proteger claves privadas y cifrar datos sensibles en tránsito y en reposo.

7. Manejo y Registro de Errores

Registrar los errores sin exponer información sensible y asegurarse de que los mensajes al usuario no revelen detalles internos del sistema.

8. Protección de Datos y Seguridad de las Comunicaciones

Cifrar todos los datos sensibles y proteger las comunicaciones con protocolos seguros como TLS para garantizar la confidencialidad e integridad.

9. Configuración del Sistema

Limitar los permisos del sistema al mínimo necesario, deshabilitar servicios no utilizados y mantener los sistemas actualizados para mitigar riesgos.

10. Gestión de Archivos y Memoria

Evitar condiciones de carrera y errores de memoria al inicializar variables, sincronizar recursos compartidos y gestionar adecuadamente la memoria asignada.

11. Prácticas Generales de Codificación

Utilizar código probado y seguro, evitar funciones dinámicas inseguras y validar todas las bibliotecas de terceros antes de integrarlas en el proyecto.